

Modularización y Funciones

Cuando un programa empieza a crecer, escribir todo dentro de `main` se vuelve una mala idea.

Sí, puede funcionar para ejemplos mínimos. Pero cuando el problema crece, necesitás dividir el programa en partes más pequeñas, comprensibles y reutilizables.

Ahí aparece la **modularización**.

En esta lección vas a aprender:

- qué es modularizar
- qué es un subprograma
- qué diferencia conceptual hay entre función y procedimiento
- cómo se declara y usa una función en C
- por qué modularizar mejora el diseño del programa

Idea clave: modularizar es dividir un problema grande en partes más pequeñas para poder pensarlo, implementarlo y mantenerlo mejor.

¿Qué es modularización?

Modularizar significa dividir un programa en partes más pequeñas y organizadas, donde cada una cumple una tarea específica.

En vez de tener un bloque enorme de código mezclando todo, separás responsabilidades.

Por ejemplo, en lugar de escribir un programa gigante que:

- lea datos
- valide datos
- calcule resultados
- muestre resultados

mezclando todo en un solo bloque, podés separar cada tarea en una parte distinta.

¿Por qué modularizar?

Porque mejora muchas cosas a la vez:

- el código se entiende mejor
- cada parte tiene una responsabilidad más clara
- podés reutilizar lógica
- es más fácil detectar errores
- el programa crece con más orden

¿Qué es un subprograma?

Un **subprograma** es una parte del programa que realiza una tarea específica.

Es, básicamente, un bloque de código con nombre propio al que el programa puede acudir cuando necesita ejecutar esa tarea.

En C, los subprogramas se expresan mediante **funciones**.

Funciones y procedimientos: diferencia conceptual

Acá hay una distinción MUY útil a nivel pedagógico.

Función

Una **función** es un subprograma que devuelve un resultado.

Por ejemplo, una función que suma dos números y devuelve el total.

Procedimiento

Un **procedimiento** es un subprograma cuya idea principal es realizar una acción, sin centrarse en devolver un valor.

Por ejemplo, mostrar un mensaje en pantalla.

Importante en C

En C, técnicamente todo se expresa con funciones.

O sea: incluso aquello que conceptualmente llamamos “procedimiento” suele escribirse como una función con retorno `void`.

Entonces, la diferencia entre función y procedimiento en este curso se enseña como una **distinción conceptual**, no como dos mecanismos distintos del lenguaje.

Ejemplo de función que devuelve un valor

```
int sumar(int a, int b) {  
    return a + b;  
}
```

Esta función recibe dos enteros y devuelve otro entero.

Ejemplo de procedimiento conceptual en C

```
void mostrarSaludo() {  
    printf("Hola\n");  
}
```

Acá no se devuelve un valor útil. La función realiza una acción.

Estructura general de una función

```
tipo_retorno nombre(parametros) {  
    instrucciones;  
}
```

Partes

- **tipo de retorno:** qué devuelve la función
- **nombre:** cómo se llama
- **parámetros:** datos que recibe
- **cuerpo:** instrucciones que ejecuta

Ejemplo completo

```
#include <stdio.h>

int sumar(int a, int b) {
    return a + b;
}

void mostrarResultado(int resultado) {
    printf("El resultado es %d\n", resultado);
}

int main() {
    int total;

    total = sumar(5, 3);
    mostrarResultado(total);

    return 0;
}
```

¿Qué se gana con esto?

Muchísimo.

Porque ahora la lógica está separada:

- una parte suma
- otra parte muestra
- `main` coordina

Eso hace que el programa se lea mucho mejor.

Modularización en varios archivos

Cuando el programa crece más, la modularización también puede reflejarse en distintos archivos.

De manera general, aparecen:

- archivos `.h` para declaraciones
- archivos `.c` para implementaciones
- un `main.c` como punto de entrada

Ejemplo conceptual

Archivo de cabecera

```
int sumar(int a, int b);
```

Archivo de implementación

```
int sumar(int a, int b) {  
    return a + b;  
}
```

¿Cuándo conviene crear un subprograma?

Conviene cuando:

- una tarea tiene sentido por sí misma
- una lógica se repite
- un bloque dentro de `main` ya está creciendo demasiado
- querés separar responsabilidades

Errores comunes al empezar

1. Meter todo en `main`

Eso vuelve el código difícil de leer y mantener.

2. Crear funciones sin una responsabilidad clara

No se trata de dividir por dividir. Cada subprograma debe tener una intención concreta.

3. No distinguir entre “calcular” y “mostrar”

Una buena modularización separa cálculo, entrada y salida cuando tiene sentido.

Resumen

- modularizar es dividir un programa en partes más pequeñas
- un subprograma realiza una tarea específica
- una función devuelve un valor
- un procedimiento, conceptualmente, realiza una acción
- en C, ambos se representan mediante funciones
- modularizar mejora claridad, reutilización y mantenimiento

Idea final

La modularización es una señal de madurez al programar.

Cuando dejás de pensar el programa como un bloque gigante y empezás a pensarlo como piezas con responsabilidades claras, tu diseño mejora MUCHÍSIMO.

Variables y Parámetros

Una vez que empezás a trabajar con funciones, aparece una pregunta central:

¿qué datos conoce cada función y cómo se le pasan esos datos?

Acá entran en juego las variables locales, las variables globales y el pasaje de parámetros.

En esta lección vas a aprender:

- qué son variables locales y globales
- qué son parámetros nominales y efectivos
- qué es el pasaje por valor
- qué es el pasaje por referencia
- cómo cambian los datos dentro y fuera de una función

Variables locales

Una **variable local** es una variable declarada dentro de una función.

Solo existe dentro de esa función y solo ahí puede usarse directamente.

Ejemplo

```
void saludar() {  
    int veces = 1;  
    printf("Hola %d\n", veces);  
}
```

La variable `veces` es local a `saludar`.

Variables globales

Una **variable global** se declara fuera de todas las funciones.

Puede ser utilizada por varias funciones del programa.

Ejemplo

```
int contador = 0;  
  
void aumentar() {  
    contador = contador + 1;  
}
```

Diferencia conceptual

- la variable local pertenece a una función
- la variable global pertenece al programa completo o a una región mucho más amplia

¿Conviene usar variables globales?

Pueden ser útiles en algunos casos, pero hay que usarlas con criterio.

¿Por qué? Porque si muchas funciones modifican una misma variable global, el programa puede volverse más difícil de entender y controlar.

Para principiantes, una buena regla es esta:

preferiré variables locales salvo que haya una razón clara para compartir el dato.

¿Qué es un parámetro?

Un **parámetro** es un dato que una función recibe para poder trabajar.

Ejemplo

```
int sumar(int a, int b) {  
    return a + b;  
}
```

Acá, a y b son parámetros de la función.

Parámetros nominales y efectivos

Esta distinción es importantísima.

Parámetros nominales

Son los nombres que aparecen en la definición de la función.

```
int sumar(int a, int b)
```

Acá, a y b son parámetros nominales.

Parámetros efectivos

Son los valores o variables reales que se pasan cuando la función se invoca.

```
resultado = sumar(5, 3);
```

Acá, 5 y 3 son parámetros efectivos.

Pasaje por valor

En el **pasaje por valor**, la función recibe una copia del dato.

Eso significa que si la función modifica ese parámetro, el original no cambia.

Ejemplo

```
#include <stdio.h>  
  
void cambiar(int x) {  
    x = 100;  
}  
  
int main() {  
    int numero = 5;  
    cambiar(numero);  
    printf("%d\n", numero);  
    return 0;  
}
```

El valor mostrado seguirá siendo 5.

¿Y por qué? Porque `cambiar` recibió una copia de `numero`, no el original.

Pasaje por referencia

En C no existe “pasaje por referencia” nativo como en otros lenguajes, pero se puede lograr el mismo efecto práctico usando **punteros**.

Esto es importante decirlo bien para no enseñar mal.

Cuando en cursos introductorios se habla de “pasaje por referencia” en C, normalmente se está hablando de pasar la dirección de una variable mediante punteros, para que la función pueda modificar el dato original.

Ejemplo conceptual

```
void cambiar(int *x) {  
    *x = 100;  
}
```

Y la llamada:

```
cambiar(&numero);
```

¿Qué cambia acá?

Ahora la función no trabaja sobre una simple copia del valor, sino sobre la dirección del dato original.

Eso permite modificar la variable real.

Ejemplo completo

```
#include <stdio.h>  
  
void cambiar(int *x) {  
    *x = 100;  
}  
  
int main() {  
    int numero = 5;  
    cambiar(&numero);  
    printf("%d\n", numero);  
    return 0;  
}
```

Ahora el valor mostrado será 100.

Comparación entre ambos pasajes

Por valor

- la función recibe una copia
- el original no cambia

Por referencia simulada con punteros

- la función trabaja sobre la dirección del dato

- el original sí puede cambiar

¿Cuándo conviene cada uno?

Pasaje por valor

Conviene cuando solo querés usar el dato para calcular o consultar algo.

Pasaje por referencia con punteros

Conviene cuando querés que la función pueda modificar el dato original.

Errores comunes al empezar

1. Creer que cambiar un parámetro por valor modifica el original

No. Si se pasó por valor, lo que se modifica es la copia.

2. Usar demasiadas variables globales

Eso vuelve más difícil entender de dónde salen y quién modifica los datos.

3. Confundir parámetros nominales con efectivos

Uno pertenece a la definición de la función. El otro pertenece a la llamada.

Resumen

- una variable local vive dentro de una función
- una variable global tiene alcance más amplio
- los parámetros nominales aparecen en la definición
- los parámetros efectivos aparecen en la invocación
- por valor se pasa una copia
- en C, el efecto de pasaje por referencia se logra usando punteros

Idea final

Entender alcance y pasaje de parámetros te cambia por completo la forma de leer funciones.

Porque a partir de acá ya no solo importa qué hace una función, sino también **con qué datos trabaja y qué datos puede llegar a modificar**.

Recursividad

La recursividad es una de esas ideas que al principio parecen confusas o “mágicas”, pero cuando lográs visualizar qué pasa detrás de escena en la memoria de la computadora, te cambia la forma de programar para siempre.

En su definición más simple: **una función recursiva es una función que se llama a sí misma**.

En esta lección vas a aprender:

- Qué es la recursividad a través de una analogía sencilla del mundo real.
 - Las dos reglas obligatorias de toda función recursiva (Caso Base y Paso Recursivo).
 - Qué es el **Call Stack (Pila de llamadas)** y cómo funciona la memoria RAM durante la recursión.
 - Qué es un **Stack Overflow** (y cómo evitar que tu programa se rompa).
 - Ejemplos prácticos en C: Factorial y Fibonacci analizados paso a paso.
 - Cuándo usar recursividad y cuándo es mejor usar un simple ciclo (**for** o **while**).
-

La Analogía de las Cajas Anidadas

Imaginá que te regalan una caja de madera grande. Al abrirla, descubrís que adentro hay otra caja más chica. Y adentro de esa, otra más chica. Te dicen que en alguna de todas las cajas anidadas está la **llave del tesoro**.

¿Cómo harías un algoritmo para encontrarla? Tenés dos formas de resolverlo:

Opción 1: El enfoque iterativo (Ciclos)

1. Armás una pila con las cajas que vas encontrando.
2. Mientras la pila no esté vacía:
 - Agarrás una caja.
 - Si adentro hay otra caja, la sumás a tu pila.
 - Si adentro está la llave, ¡terminaste!

Opción 2: El enfoque recursivo (Llamarse a uno mismo)

1. Mirás el contenido de la caja actual.
2. Si encontrás otra caja, **hacés exactamente lo mismo** (llamar a la función de búsqueda sobre esa nueva caja).
3. Si encontrás la llave, ¡terminaste!

```
Función BuscarLlave(Caja)
1. Abrir Caja
2. Si lo que hay adentro es la LLAVE -> ¡Terminar! (Caso Base)
3. Si lo que hay adentro es otra CAJA -> BuscarLlave(NuevaCaja) (Paso Recursivo)
```

En la opción recursiva, no necesitás llevar una lista manual de las cajas que te quedan por revisar. La computadora se encarga de recordar en qué caja estabas de manera automática.

Las Dos Reglas de Oro de la Recursividad

Si escribís una función que se llama a sí misma sin control, vas a congelar la computadora. Toda función recursiva correcta debe cumplir dos reglas estrictas:

1. **El Caso Base (La salida de emergencia):** Es la condición que le dice a la función cuándo debe detenerse y empezar a devolver valores. Sin esto, la función se llamaría infinitamente.

2. **El Paso Recursivo (Acercarse a la salida):** Es la llamada a la función a sí misma, pero **con un argumento modificado** que esté más cerca del caso base. Si llamás a la función siempre con el mismo valor, nunca vas a llegar a la condición de parada.
-

¿Qué pasa en la memoria? El Call Stack (Pila de llamadas)

Para entender la recursividad, tenés que entender el **Call Stack** (Pila de llamadas).

Cuando tu programa ejecuta una función, la computadora reserva un bloque de memoria (llamado *Stack Frame* o Marco de Pila) para almacenar las variables locales y los parámetros de esa función. Este bloque se coloca arriba de una pila, como si fueran platos.

- Si la función A llama a la función B, el plato de B se pone arriba del de A.
- La computadora solo puede trabajar con el plato que está **arriba del todo**.
- Cuando la función B termina, su plato se retira y la computadora vuelve a la función A justo donde la había dejado.

La traza de memoria con Factorial

El factorial de un número N (escrito como $N!$) es el producto de todos los números desde 1 hasta N . Por ejemplo, $3! = 3 \times 2 \times 1 = 6$.

Veamos la implementación en C:

```
int factorial(int n) {
    // 1. Caso Base: Si n es 1, no hace falta calcular nada más
    if (n == 1) {
        return 1;
    }
    // 2. Paso Recursivo: n es mayor a 1, multiplicamos n por el factorial de
    // (n-1)
    else {
        return n * factorial(n - 1);
    }
}
```

Si llamamos a `factorial(3)`, mirá cómo se van apilando las funciones en el Call Stack:

```
Paso 1: Llamamos a factorial(3)
[ factorial(3) -> espera que factorial(2) responda ] <-- Cima del Stack

Paso 2: factorial(3) llama a factorial(2)
[ factorial(2) -> espera que factorial(1) responda ] <-- Cima del Stack
[ factorial(3) -> esperando... ]

Paso 3: factorial(2) llama a factorial(1)
[ factorial(1) -> ¡Caso base! Devuelve 1 ] <-- Cima del Stack
[ factorial(2) -> esperando... ]
[ factorial(3) -> esperando... ]
```

Una vez que llegamos al caso base (`factorial(1)` que devuelve `1`), la pila empieza a **desapilarse** y a resolver las operaciones pendientes de arriba hacia abajo:

```
Paso 4: factorial(1) retorna 1. factorial(2) calcula: 2 * 1 = 2
[ factorial(2) -> retorna 2 ] <-- Cima del Stack
```

```
[ factorial(3) -> esperando... ]
```

```
Paso 5: factorial(2) retorna 2. factorial(3) calcula: 3 * 2 = 6  
[ factorial(3) -> retorna 6 ]                                <-- Cima del Stack
```

Resultado Final: 6. La pila queda vacía.

El peligro número uno: Stack Overflow (Desbordamiento de Pila)

¿Qué pasa si nos olvidamos del caso base o si el paso recursivo no se acerca a él?

```
// ¡CUIDADO! Recursión infinita  
void funcionInfinita() {  
    funcionInfinita();  
}
```

Cada llamada a `funcionInfinita()` va a poner un nuevo plato en la pila de memoria sin retirar los anteriores. Dado que la memoria física de la computadora es limitada, llegará un momento en el que la pila se llenará por completo.

Cuando esto ocurre, el sistema operativo interrumpe el programa lanzando un error fatal: **Stack Overflow** (Desbordamiento de Pila). Tu programa se cerrará de golpe.

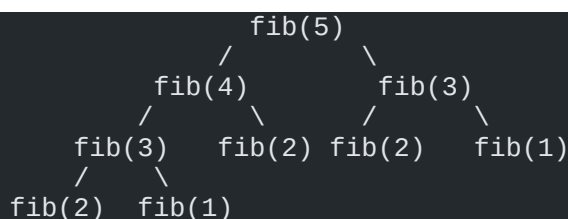
Ejemplo Avanzado: La sucesión de Fibonacci

La famosa sucesión de Fibonacci empieza con \$0\$ y \$1\$, y cada término siguiente es la suma de los dos anteriores: \$0, 1, 1, 2, 3, 5, 8, 13...\$

```
int fibonacci(int n) {  
    // Casos base  
    if (n == 0) return 0;  
    if (n == 1) return 1;  
  
    // Paso recursivo (doble llamada)  
    return fibonacci(n - 1) + fibonacci(n - 2);  
}
```

¿Por qué Fibonacci recursivo es ineficiente?

Si llamas a `fibonacci(5)`, la función se divide en dos llamadas (`fibonacci(4)` y `fibonacci(3)`), y estas a su vez en otras dos, creando un **árbol de llamadas**:



Fijate que `fibonacci(3)` se calcula **2 veces**, y `fibonacci(2)` se calcula **3 veces** de forma completamente independiente. Para números grandes (como `fibonacci(50)`), la computadora tendrá que realizar miles de millones de cálculos duplicados y tardará minutos o incluso horas en responder.

Recursividad vs. Iteración: ¿Cuál elegir?

Criterio	Recursividad	Iteración (Ciclos <code>for/while</code>)
Estructura	Código elegante, limpio y más fácil de leer en problemas jerárquicos (como árboles o gráficos).	Requiere más variables de control manuales, pero el código es muy secuencial.
Memoria	Costo Alto. Consume espacio en el Call Stack por cada llamada recursiva.	Costo Bajo. Utiliza una cantidad constante de memoria independientemente del número de repeticiones.
Velocidad	Más lenta debido al costo de crear y destruir marcos de pila en memoria.	Más rápida, ya que son saltos de instrucciones directos en el procesador.

Regla general:

- Usa **iteración** para tareas simples y lineales (recorrer un arreglo, contar números, acumuladores).
 - Usa **recursividad** cuando estés trabajando con estructuras que se definen recursivamente (como árboles binarios, carpetas del sistema) o algoritmos de ordenación avanzados (*Merge Sort*, *Quick Sort*).
-

Resumen de Aprendizajes

- La recursividad consiste en resolver un problema dividiéndolo en versiones más pequeñas de sí mismo.
- **Caso Base:** Detiene la función y evita un Stack Overflow.
- **Paso Recursivo:** Vuelve a llamar a la función modificando el argumento.
- La memoria de la computadora usa un **Call Stack** para recordar dónde iba cada función suspendida.
- La elegancia de la recursividad tiene un costo en velocidad y uso de memoria.

Árboles Binarios

Hasta ahora, aprendiste a organizar tus datos en estructuras **lineales**: listas, pilas y colas. En ellas, cada elemento tiene un único sucesor (como una fila de supermercado). Pero el mundo real no siempre es una fila recta. A veces los datos se organizan de forma **jerárquica**, como el organigrama de una empresa, los archivos en tu computadora o las ramas de un árbol.

Ahí es donde entra el **árbol binario**.

En esta lección vas a aprender:

- Qué es un árbol binario y por qué es tan poderoso.
 - La anatomía de un árbol (raíz, hojas, subárboles).
 - Cómo definir un nodo en C utilizando estructuras y punteros.
 - Qué es un Árbol Binario de Búsqueda (BST) y cómo organizar datos en él.
 - Cómo insertar, buscar y recorrer elementos utilizando la magia de la **recursividad**.
-

¿Por qué necesitamos Árboles Binarios?

Imaginá que tenés una lista enlazada con 1.000.000 de números ordenados de menor a mayor y querés buscar el número 999.999. Vas a tener que recorrer casi todo el millón de nodos, uno por uno, para encontrarlo. ¡Eso es lento!

Un **árbol binario de búsqueda** nos permite partir el problema a la mitad en cada paso. Es como buscar una palabra en el diccionario: si abrís el libro a la mitad y ves que la palabra empieza con ‘Z’, descartás toda la mitad izquierda inmediatamente. En lugar de hacer un millón de preguntas, en un árbol bien balanceado podés encontrar tu dato en aproximadamente **20 pasos**.

Anatomía de un Árbol

Visualizá un árbol al revés (con la raíz arriba y las ramas creciendo hacia abajo):

```
      [ Raíz: 50 ]      <-- El nodo principal, donde todo empieza
      /      \
    [ 30 ]    [ 70 ]    <-- Nodos internos (tienen padre e hijos)
    /  \    /  \
  [20] [40] [60] [80]  <-- Hojas (nodos finales, no tienen hijos)
```

Conceptos clave para recordar:

- **Raíz (Root):** El nodo superior del árbol. No tiene “padre”.
 - **Hijo (Child):** Nodo que cuelga de otro superior. En un árbol **binario**, cada nodo puede tener como máximo **dos hijos** (hijo izquierdo e hijo derecho).
 - **Padre (Parent):** Nodo que tiene ramas hacia nodos inferiores.
 - **Hoja (Leaf):** Nodos que no tienen hijos (sus punteros izquierdo y derecho apuntan a NULL).
 - **Subárbol (Subtree):** Cada hijo de un nodo puede considerarse a su vez la raíz de su propio árbol más chico. ¡Esta es la base de la recursividad!
-

Definiendo un Nodo en C

Para construir esta estructura en C, necesitamos definir un `struct` que guarde el valor (el dato) y dos punteros: uno para el hijo izquierdo y otro para el hijo derecho.

```
#include <stdio.h>
#include <stdlib.h> // Para usar malloc y free

// Definición de la estructura de un Nodo
struct Nodo {
```

```
int dato; // El valor que almacena el nodo
struct Nodo* izquierdo; // Puntero al subárbol izquierdo
struct Nodo* derecho; // Puntero al subárbol derecho
};
```

[!NOTE] Prestá atención a esto: `struct Nodo*` es un puntero a otro nodo del mismo tipo. Es una estructura autorreferenciada. Así es como logramos enlazar los nodos entre sí.

Creación de un Nuevo Nodo

Para agregar elementos al árbol, primero debemos poder crear nodos en la memoria del sistema (el *Heap*) usando `malloc`.

```
// Función auxiliar para crear un nuevo nodo
struct Nodo* crearNodo(int valor) {
    // 1. Reservamos memoria para el nodo
    struct Nodo* nuevoNodo = (struct Nodo*)malloc(sizeof(struct Nodo));

    // 2. Asignamos el valor
    nuevoNodo->dato = valor;

    // 3. Inicializamos los hijos como vacíos (NULL)
    nuevoNodo->izquierdo = NULL;
    nuevoNodo->derecho = NULL;

    return nuevoNodo;
}
```

La Regla de Oro: Árbol Binario de Búsqueda (BST)

Un árbol binario común y corriente no tiene un orden establecido. Pero para buscar datos rápido, usamos el **Árbol Binario de Búsqueda (BST)**. Su regla es muy sencilla y estricta:

Para cualquier nodo del árbol:

1. Todos los valores en su **subárbol izquierdo** deben ser **menores** que el valor del nodo.
2. Todos los valores en su **subárbol derecho** deben ser **mayores** que el valor del nodo.

Inserción de Datos

Para insertar un elemento respetando esta regla, usamos **recursividad**:

```
// Función para insertar un valor en el árbol
struct Nodo* insertar(struct Nodo* raiz, int valor) {
    // Caso base: si el árbol (o subárbol) está vacío, creamos el nodo ahí
    if (raiz == NULL) {
        return crearNodo(valor);
    }

    // Si el valor es menor, va al subárbol izquierdo
    if (valor < raiz->dato) {
        raiz->izquierdo = insertar(raiz->izquierdo, valor);
    }
}
```

```
// Si el valor es mayor, va al subárbol derecho
else if (valor > raiz->dato) {
    raiz->derecho = insertar(raiz->derecho, valor);
}

// Devolvemos el puntero del nodo (sin cambios)
return raiz;
}
```

Búsqueda de un Elemento

Buscar en un BST es sumamente eficiente porque en cada nodo decidimos si ir a la izquierda o a la derecha, descartando la otra mitad del árbol.

```
// Función para buscar un valor en el árbol
struct Nodo* buscar(struct Nodo* raiz, int valorBuscado) {
    // Caso base: si el árbol está vacío o si encontramos el valor
    if (raiz == NULL || raiz->dato == valorBuscado) {
        return raiz;
    }

    // Si el valor es menor, buscamos en la izquierda
    if (valorBuscado < raiz->dato) {
        return buscar(raiz->izquierdo, valorBuscado);
    }

    // Si el valor es mayor, buscamos en la derecha
    return buscar(raiz->derecho, valorBuscado);
}
```

Recorridos (Cómo leer el árbol)

En una lista lineal, lees de principio a fin. En un árbol, tenés distintas formas de visitarlo. Las tres principales son recursivas:

1. **Preorden (Pre-order):** Visita primero la raíz, luego el subárbol izquierdo y después el derecho. (Raíz -> Izquierda -> Derecha).
2. **Inorden (In-order):** Visita primero el subárbol izquierdo, luego la raíz y después el derecho. (Izquierda -> Raíz -> Derecha).

[!TIP] ¡En un BST, el recorrido **inorden** siempre te va a mostrar los números ordenados de menor a mayor!

3. **Postorden (Post-order):** Visita primero el subárbol izquierdo, luego el derecho y por último la raíz. (Izquierda -> Derecha -> Raíz).

Implementación del Recorrido Inorden en C

```
// Función para imprimir el árbol en inorden
void inorden(struct Nodo* raiz) {
    if (raiz != NULL) {
        inorden(raiz->izquierdo); // 1. Recorrer subárbol izquierdo
        printf("%d ", raiz->dato); // 2. Visitar el nodo actual (raíz)
    }
}
```

```
        inorden(raiz->derecho);        // 3. Recorrer subárbol derecho
    }
}
```

Un Ejemplo Completo para Probar

Aquí tenés un programa de C completo y funcional que podés compilar y ejecutar. Este programa crea un árbol, inserta algunos valores, los imprime ordenados y realiza una búsqueda:

```
#include <stdio.h>
#include <stdlib.h>

struct Nodo {
    int dato;
    struct Nodo* izquierdo;
    struct Nodo* derecho;
};

struct Nodo* crearNodo(int valor) {
    struct Nodo* nuevoNodo = (struct Nodo*)malloc(sizeof(struct Nodo));
    nuevoNodo->dato = valor;
    nuevoNodo->izquierdo = NULL;
    nuevoNodo->derecho = NULL;
    return nuevoNodo;
}

struct Nodo* insertar(struct Nodo* raiz, int valor) {
    if (raiz == NULL) return crearNodo(valor);

    if (valor < raiz->dato) {
        raiz->izquierdo = insertar(raiz->izquierdo, valor);
    } else if (valor > raiz->dato) {
        raiz->derecho = insertar(raiz->derecho, valor);
    }
    return raiz;
}

void inorden(struct Nodo* raiz) {
    if (raiz != NULL) {
        inorden(raiz->izquierdo);
        printf("%d ", raiz->dato);
        inorden(raiz->derecho);
    }
}

struct Nodo* buscar(struct Nodo* raiz, int valor) {
    if (raiz == NULL || raiz->dato == valor) return raiz;
    if (valor < raiz->dato) return buscar(raiz->izquierdo, valor);
    return buscar(raiz->derecho, valor);
}

int main() {
    struct Nodo* raiz = NULL;

    // Insertamos elementos
    raiz = insertar(raiz, 50);
    insertar(raiz, 30);
    insertar(raiz, 70);
    insertar(raiz, 20);
    insertar(raiz, 40);
```



```

insertar(raiz, 60);
insertar(raiz, 80);

// Imprimimos el recorrido inorden (debe salir ordenado de menor a mayor)
printf("Elementos en el arbol (recorrido Inorden): ");
inorden(raiz);
printf("\n");

// Buscamos un elemento
int numeroABuscar = 40;
struct Nodo* encontrado = buscar(raiz, numeroABuscar);
if (encontrado != NULL) {
    printf("El numero %d fue encontrado en el arbol.\n", numeroABuscar);
} else {
    printf("El numero %d no existe en el arbol.\n", numeroABuscar);
}

return 0;
}

```

Listas

Las **listas enlazadas** son estructuras dinámicas. Eso significa que pueden crecer o cambiar durante la ejecución del programa.

En esta lección vas a aprender:

- qué es una lista enlazada
- qué es un nodo
- recorrido en listas
- búsqueda en listas
- inserción en listas

¿Qué es una lista enlazada?

Es una estructura formada por nodos.

Cada nodo suele contener:

- un dato
- un puntero al siguiente nodo

Nodo en C

```

struct Nodo {
    int dato;
    struct Nodo* siguiente;
};

```

Recorrido

Recorrer una lista significa visitar nodo por nodo hasta llegar al final.

```

actual = cabeza;

```

```
while (actual != NULL) {
    printf("%d\n", actual->dato);
    actual = actual->siguiente;
}
```

Búsqueda

Buscar en una lista implica recorrerla hasta encontrar el valor buscado.

```
while (actual != NULL) {
    if (actual->dato == buscado) {
        encontrado = 1;
    }
    actual = actual->siguiente;
}
```

Inserción

Insertar puede hacerse al inicio, al final o en una posición intermedia.

Inserción al inicio

```
nuevo->siguiente = cabeza;
cabeza = nuevo;
```

Resumen

- una lista enlazada está formada por nodos
- permite recorrido, búsqueda e inserción
- es más flexible que un arreglo de tamaño fijo

Idea final

La lista enlazada muestra un cambio de mentalidad muy fuerte: ya no pensás en posiciones fijas, sino en enlaces entre nodos.

Lecciones 22 / 32

Pilas

Una **pila** es una estructura en la que el último elemento que entra es el primero que sale.

Eso se conoce como **LIFO**: Last In, First Out.

En esta lección vas a aprender:

- qué es una pila
- recorrido en pilas
- búsqueda en pilas
- inserción en pilas
- extracción básica

Inserción: push

Agregar un elemento a la pila se conoce como push.

Extracción: pop

Quitar el elemento superior se conoce como pop.

Recorrido

El recorrido visita los elementos desde el tope hacia abajo.

Búsqueda

La búsqueda se hace recorriendo la pila elemento por elemento.

Ejemplo de nodo

```
struct Nodo {  
    int dato;  
    struct Nodo* siguiente;  
};
```

Resumen

- una pila sigue la regla LIFO
- insertar se llama push
- extraer se llama pop
- también puede recorrerse y buscarse

Idea final

La pila es excelente para entender que no todas las estructuras permiten acceder a los datos de cualquier forma. Cada una tiene su propia lógica de uso.

Colas

Una **cola** es una estructura en la que el primero que entra es el primero que sale.

Eso se conoce como **FIFO**: *First In, First Out*.

Pensalo como una fila real:

- la primera persona en llegar es la primera en ser atendida
- las nuevas personas se agregan al final
- nadie sale por el medio

En esta lección vas a aprender:

- qué es una cola
- recorrido en colas
- búsqueda en colas
- inserción en colas
- extracción básica

Idea clave: una cola respeta estrictamente el orden de llegada.

¿Qué es una cola?

Una **cola** es una estructura dinámica donde normalmente se manejan dos extremos:

- el **frente**
- el **final**

La inserción ocurre al final. La extracción ocurre desde el frente.

Nodo de una cola enlazada

Una forma común de implementar una cola en C es mediante nodos enlazados.

```
struct Nodo {  
    int dato;  
    struct Nodo* siguiente;  
};
```

Cada nodo tiene:

- un dato
- un puntero al siguiente nodo

Variables principales de la cola

Para manejar la cola, normalmente se usan dos punteros:

```
struct Nodo* frente = NULL;  
struct Nodo* final = NULL;
```

- **frente** apunta al primer nodo
- **final** apunta al último nodo

Si ambos valen NULL, la cola está vacía.

Inserción en colas: encolar

Agregar un elemento al final de la cola se llama **encolar**.

Idea general

Si la cola está vacía:

- el nuevo nodo pasa a ser frente y final

Si la cola ya tiene elementos:

- el nodo actual final debe apuntar al nuevo nodo
- luego `final` pasa a apuntar al nuevo nodo

Ejemplo de código

```
#include <stdlib.h>

void encolar(struct Nodo** frente, struct Nodo** final, int valor) {
    struct Nodo* nuevo = malloc(sizeof(struct Nodo));

    nuevo->dato = valor;
    nuevo->siguiente = NULL;

    if (*final == NULL) {
        *frente = nuevo;
        *final = nuevo;
    } else {
        (*final)->siguiente = nuevo;
        *final = nuevo;
    }
}
```

¿Qué hace esta función?

1. crea un nuevo nodo
2. guarda el valor en ese nodo
3. si la cola está vacía, ese nodo pasa a ser el primero y el último
4. si no está vacía, se enlaza al final actual y luego pasa a ser el nuevo final

Extracción básica: desencolar

Quitar el elemento del frente se llama **desencolar**.

Idea general

Cuando desencolás:

- se toma el nodo del frente
- el frente avanza al siguiente nodo
- si la cola queda vacía, también hay que actualizar `final`

Ejemplo de código

```
#include <stdlib.h>

int desencolar(struct Nodo** frente, struct Nodo** final) {
    struct Nodo* temp;
    int valor;

    if (*frente == NULL) {
        return -1;
    }

    temp = *frente;
    valor = temp->dato;
    *frente = (*frente)->siguiente;
```

```
if (*frente == NULL) {
    *final = NULL;
}

free(temp);
return valor;
}
```

¿Qué hace esta función?

1. verifica si la cola está vacía
2. guarda el dato del frente
3. mueve `frente` al siguiente nodo
4. si ya no queda ningún nodo, también pone `final` en `NULL`
5. libera memoria del nodo eliminado
6. devuelve el valor extraído

Recorrido en colas

El **recorrido** visita los nodos desde el frente hasta el final.

Ejemplo de código

```
#include <stdio.h>

void recorrerCola(struct Nodo* frente) {
    struct Nodo* actual = frente;

    while (actual != NULL) {
        printf("%d\n", actual->dato);
        actual = actual->siguiente;
    }
}
```

¿Qué hace este recorrido?

- empieza en el frente
- muestra el dato de cada nodo
- avanza hasta llegar a `NULL`

Si la cola contiene:

```
10, 20, 30
```

el recorrido mostrará:

```
10
20
30
```

Búsqueda en colas

La **búsqueda** consiste en recorrer la cola hasta encontrar un valor.

Ejemplo de código

```
int buscarEnCola(struct Nodo* frente, int buscado) {
    struct Nodo* actual = frente;

    while (actual != NULL) {
        if (actual->dato == buscado) {
            return 1;
        }
        actual = actual->siguiente;
    }

    return 0;
}
```

¿Qué hace esta función?

- recorre nodo por nodo
- si encuentra el valor, devuelve 1
- si llega al final sin encontrarlo, devuelve 0

Ejemplo completo de uso

```
#include <stdio.h>
#include <stdlib.h>

struct Nodo {
    int dato;
    struct Nodo* siguiente;
};

void encolar(struct Nodo** frente, struct Nodo** final, int valor) {
    struct Nodo* nuevo = malloc(sizeof(struct Nodo));

    nuevo->dato = valor;
    nuevo->siguiente = NULL;

    if (*final == NULL) {
        *frente = nuevo;
        *final = nuevo;
    } else {
        (*final)->siguiente = nuevo;
        *final = nuevo;
    }
}

int desencolar(struct Nodo** frente, struct Nodo** final) {
    struct Nodo* temp;
    int valor;

    if (*frente == NULL) {
        return -1;
    }

    temp = *frente;
    valor = temp->dato;
    *frente = (*frente)->siguiente;

    if (*frente == NULL) {
        *final = NULL;
    }
}
```

```

    free(temp);
    return valor;
}

void recorrerCola(struct Nodo* frente) {
    struct Nodo* actual = frente;

    while (actual != NULL) {
        printf("%d\n", actual->dato);
        actual = actual->siguiente;
    }
}

int buscarEnCola(struct Nodo* frente, int buscado) {
    struct Nodo* actual = frente;

    while (actual != NULL) {
        if (actual->dato == buscado) {
            return 1;
        }
        actual = actual->siguiente;
    }

    return 0;
}

int main() {
    struct Nodo* frente = NULL;
    struct Nodo* final = NULL;
    int eliminado;

    encolar(&frente, &final, 10);
    encolar(&frente, &final, 20);
    encolar(&frente, &final, 30);

    printf("Recorrido de la cola:\n");
    recorrerCola(frente);

    if (buscarEnCola(frente, 20) == 1) {
        printf("Se encontró el valor 20\n");
    } else {
        printf("No se encontró el valor 20\n");
    }

    eliminado = desencolar(&frente, &final);
    printf("Valor extraído: %d\n", eliminado);

    printf("Cola luego de desencolar:\n");
    recorrerCola(frente);

    return 0;
}

```

Resumen

- una cola sigue la regla FIFO
- insertar se llama **encolar**
- extraer se llama **desencolar**
- el recorrido va del frente al final

- la búsqueda recorre nodo por nodo hasta encontrar el valor
- al desencolar hay que actualizar correctamente **frente** y, si hace falta, también **final**

Idea final

La cola no solo sirve para entender una estructura dinámica: también enseña una idea de diseño muy importante.

No siempre se puede acceder a los datos de cualquier manera. A veces la estructura define una disciplina de acceso. Y entender esa disciplina es parte de aprender a programar bien.